CogniFlow - Demo Video Guide

Everything you need to record the 3-5 minute demo video for the Agentic AI Hackathon, Tech Zephyr 4.0. Written for Team BloodCoded.

You are the only speaker

This video has one narrator, and that is you. You will be doing two jobs at once - driving the terminal and explaining what appears. That is harder than it sounds, so this guide is built around making it manageable: learn one idea properly, know where each line comes from, and read the script for the rest.

There are two documents. This one teaches you the concepts and tells you *when* to say things. The second - **CogniFlow_Video_Script.pdf** - contains the actual words, split into numbered PARTs. Keep both open.

How the two documents fit together

This guide (PDF 1)	The script (PDF 2)
Explains what everything means	Gives you the exact words
Tells you which PART to read and when	Is organised into PART 0 to PART 10
Read it over the next few days	Read it while recording
Sections 1-4 teach; 5-9 prepare you	Blue panels are the only things spoken

What is in this guide

Section	What it is	When to read it
0. Getting the code	Putting CogniFlow on YOUR machine	Before anything else
1. The one idea	What CogniFlow does, in plain English	First. Twice.
2. The five concepts	Every term you might be asked about	Over 2-3 sittings
3. Reading the screen	What each line of output means	With the app open
4. The map	Which PART to read at which moment	Before rehearsing
5. Narrating alone	Technique for doing both jobs at once	Before rehearsing
6. Setup	Exact commands, in order	Recording day
7. If it breaks	What to do live	Skim, then trust it
8. Questions	What judges may ask you	Before the finale
9. Glossary	Every term in one place	Whenever stuck
10. Learning plan	What to study, in priority order	Today

0. Getting the code onto your machine

You are a collaborator on the GitHub repository, but the code is not on your computer yet. Everything else in this guide assumes it is. Do this first, and do it **before** the day you plan to record - one step downloads a large file and you do not want to be watching a progress bar with the camera running.

Step 1 - Get the code

```
git clone https://github.com/sohan1611/CogniFlow.git
cd CogniFlow
```

If `git` is not recognised, install Git for Windows from git-scm.com first. If you do not have Python, install it from python.org - any version from 3.11 upwards will do.

Step 2 - Install the dependencies

```
python run.py install
```

This creates a private folder of libraries inside the project. It does not touch anything else on your computer, and takes about a minute.

Step 3 - Get your own API keys

> **Do not ask Sohan for his keys - make your own**
>
> The keys are deliberately NOT in the repository: committing credentials is forbidden by rulebook section 6 and is a disqualification path under section 13, so cloning gives you the code and no keys. Both providers we use are free, take about two minutes each, and need no billing details.
>
> Making your own is better than borrowing: you are not sharing a secret, and the two accounts have separate rate limits, so the team gets double the headroom.

Provider	Where to get a free key	Environment variable
Groq	console.groq.com -> API Keys	GROQ_API_KEY
Google	aistudio.google.com/apikey	GOOGLE_API_KEY

Then make a copy of `.env.example`, rename the copy to `.env`, and paste each key after the matching = sign. No quotes, no spaces. The file already contains the variable names and comments explaining each one.

Why both, and not just one: Groq is fast but its free tier allows about 8,000 tokens a minute, and one demo run uses roughly 35,000. So it runs out partway through and the program switches to Google automatically. With only one key configured you will see `degraded=True` on screen, which is exactly what the script tells you not to film.

Step 4 - Build the search index

```
python run.py ingest
```

This is the slow one - do it the day before

The first run downloads a **79 MB** language model used to search the course notes. It is free and it happens exactly once, but on a new machine and an ordinary connection it can take several minutes. Every later run reuses it and finishes in seconds. **Run this well before you plan to record.**

Step 5 - Prove it works

```
python run.py live
```

You want to see the line `Learning path : recursion -> functions -> recursion` and seven `[PASS]` lines at the end. If you see those, you are ready and the rest of this guide applies to you exactly as written.

Timed from a clean clone on 5 September: **97 seconds** for steps 1, 2, 4 and 5 together, on a machine that already had the 79 MB model. Add the download time for yours.

If something goes wrong

What you see	What to do
'git' is not recognised	Install Git for Windows from git-scm.com, then reopen the terminal.
'python' is not recognised	Install Python from python.org and tick 'Add Python to PATH' during setup.
Install fails on a missing package	Tell Sohan the exact error line. Do not start editing files.
degraded=True in the output	A key is missing or wrong. Check .env has both keys and no stray quotes.
It runs but is very slow the first time	That is the 79 MB download in step 4. Normal, and only once.

The fallback, if setup will not cooperate today

Sohan runs `python run.py live` on his machine and screen-records it, sends you the video file, and you record your narration over that footage. This is legitimate - section 5 of this guide already explains why narrating over a finished real run is fine, and a recording of a genuine run is the same thing. Nothing is invented. It is a worse experience for you than driving it yourself, but it is a real option and it is better than a rushed take.

1. The one idea

If you remember nothing else, remember this page. It is the entire project, and it is what PART 1 of the script says in your own voice.

The situation

A student is learning Python. They try a recursion exercise and get it wrong. They try again and get it wrong again.

What every other AI tutor does

It gives them an **easier recursion question**. Maybe it adds a hint. Maybe it explains recursion again, more slowly.

Why that fails

Very often the student's real problem is not recursion at all. It is that they do not understand what `return` does when one function calls another. That is a **functions** problem. Recursion is just where it became visible. So the easier recursion question fails too, for exactly the same reason, and the student concludes they are 'bad at recursion' when nobody ever taught them the thing recursion is built on.

What CogniFlow does instead

It notices the pattern, works out that the real gap is **functions**, **changes its own goal** to teach functions, teaches that, and then comes back to recursion - which it never forgot was the original target.

```
recursion  ->  functions  ->  recursion
(failing)      (the real      (back to what
                gap)           they wanted)
```

The sentence to memorise

"Most AI tutors personalise **which question** you get. CogniFlow personalises **the route through the knowledge**."

Everything else here is detail underneath that sentence. If a judge only hears one thing from you, this is the one.

2. The five concepts

These are the five things a judge might ask about. Each has: what it is, why it matters, and the one sentence to say. You do not need any mathematics.

2.1 The prerequisite graph (the core)

What it is. A map of which topic depends on which. You cannot understand recursion without functions. You cannot understand functions without variables. We wrote that map down as a structure the program can walk through.

```
variables  ->  functions  ->  recursion  ->  recursion_tree
variables  ->  loops      ->  nested_loops
```

Why it matters. It lets the program ask a question no chatbot can ask: 'this student keeps failing recursion - what does recursion *depend on*, and are they weak at that instead?'

Where it appears: PART 4, on the `[prereq_redirect]` line.

Say:

> *"It walks a prerequisite graph. Recursion depends on functions, so when recursion keeps failing it checks whether functions is the real problem."*

2.2 Mastery, and why it is not a score out of ten

What it is. For every skill we keep a number between 0 and 1 - our best estimate of the chance the student actually knows it. Recursion at 0.35 means "we think there is about a 35% chance they have got it".

The technique is **Bayesian Knowledge Tracing**, from a 1995 paper by Corbett and Anderson. You do not need the equations. You need to know why it beats counting right answers:

- It knows people **guess**. One lucky correct answer does not prove mastery.
- It knows people **slip**. One careless mistake does not erase what they know.
- It tracks **confidence** separately. Being 80% sure after 3 questions is not the same as after 30, and the program treats those differently.

Where it appears: PART 3 and PART 6, on the `[mastery]` lines.

Say:

> *"Mastery is a Bayesian estimate, not a running total. It models guessing and slipping, and it produces a confidence number the routing logic actually uses."*

2.3 "The model proposes, the guard disposes"

What it is. Our architecture in five words, and the thing the team is proudest of. An AI model suggests what to do next - retry, move on, go back a step. It does not get the final say. Separate, ordinary, predictable code called **the guard** checks that suggestion against fixed rules and overrules it when it breaks one.

A rule the guard enforces: *you may not send a student onward if their mastery is below 0.6.* The AI might suggest it. The guard says no.

Why it matters - three reasons, worth learning properly:

Reason	What it buys us

Safety	The AI cannot wreck someone's learning path, even if it says something strange on the day.
Measurement	We can count how often the guard overrules the AI. That is a real number in our results.
Demo reliability	Because routing is predictable code, the learning path is the same every run.

Where it appears: PART 4b - when a `[guard_override]` line shows up. It appeared in all three of our live test runs.

Say:

> *"The model proposes and the guard disposes. An LLM suggests the next action, deterministic code validates it against hard rules and overrides it when it is invalid - and we log every override."*

2.4 The safety invariant (our failure never costs the student)

What it is. Two kinds of bad thing can happen when a student submits code:

Their problem	Our problem
Their code has a syntax error	Our sandbox crashed
Their code crashes at runtime	The AI provider timed out
Their answer is wrong	Our database failed
Their code loops forever	The AI returned nonsense

The left column says something about the student, so it changes their mastery. The right column says something about **us**, so it must change nothing.

The clever part: this is not an `if` statement somebody has to remember to write. The two kinds are different *types* in the code, and the function that updates mastery only accepts the first kind. If an infrastructure failure ever reached it, the program would refuse outright.

Where it appears: PART 5, on `[recover] ... mastery_untouched=True`.

Say:

> *"Student outcomes and system faults are disjoint types, and mastery is only reachable from the first. If our infrastructure breaks, the student does not pay for it - and that is enforced by the type system, not by a conditional someone has to notice in review."*

2.5 RAG - teaching from real material

What it is. RAG stands for Retrieval-Augmented Generation. In plain terms: before the AI writes an exercise, we **search our own course notes** for the relevant section and hand it to the AI as source material.

Why it matters. Without it, the AI invents an exercise from whatever it happens to remember. With it, the exercise is grounded in the actual curriculum, and we can show which page it came from.

On screen you will see something like `05_recursion.md#5.1 The idea` - the exact section the exercise was built from.

One detail worth knowing: we do not call RAG at every step. It runs at one point out of twelve, where it helps. If asked why, the answer is that retrieval costs time and adds nothing to arithmetic.

Where it appears: PART 3, and again in PART 4 where retrieval follows the redirect into the functions chapter.

Say:

"Retrieval is filtered by skill first, then searched semantically, and every generated problem records which section it was grounded in."

3. Reading the screen

During the run, lines scroll past. Each is a real decision the program made - not a message we printed in advance.

Line you will see	What it means in plain English
[diagnose]	Working out which skill is weakest and what it depends on
[plan_action]	Choosing topic, difficulty and teaching style for the next task
[retrieve]	Searching our course notes for the right section
[generate_problem]	The AI writing the exercise, grounded in what was retrieved
[execute]	Running the student's code in a sandbox
[misconception]	Working out WHY the answer was wrong - the important one
[mastery]	Updating our estimate of what the student knows
[guard_override]	The guard rejecting the AI's suggestion - point at this
[adapt]	Deciding what to do next
[prereq_redirect]	THE MOMENT. Changing its own goal to a prerequisite
[recover]	An infrastructure failure being absorbed safely
[prereq_return]	Coming back to the original topic
[finalize]	Session finished

The three lines that matter most

- **[misconception]** - says *why* they failed, not just that they failed.
- **[prereq_redirect]** - the agent changing its own objective. The project.
- **[recover] ... mastery_untouched=True** - proof the safety invariant is real.

4. The map - which PART to read when

This is the table you rehearse from. The words are in **CogniFlow_Video_Script.pdf**.

PART	Read it when	You are doing	Length
0	Before recording anything	Setup and three test runs. Nothing spoken.	-
1	Opening shot, seeded model on screen	Pointing at functions 0.55 and recursion 0.35	30s
2	Straight after PART 1, before running anything	Just speaking. No screen action.	20s
3	Immediately after starting 'python run.py live'	Letting the first block scroll; pointing at [diagnose], [retrieve], [mastery]	45s
4	The instant [prereq_redirect] appears	STOP SCROLLING. Hold the screen still.	50s
4b	Only if a [guard_override] line appeared	Pointing at proposed= and final=	10s
5	When [execute] shows sandbox_error	Pointing at the mastery number and holding there	30s
6	When [prereq_return] appears	Scrolling to the learning path summary	25s
7	When the seven [PASS] lines are visible	Showing all seven at once	25s
8	After running 'python run.py ablation'	Showing the results table	35s
9	Final shot	Nothing. Just close.	15s
10	Only if comfortably under 5 minutes	Opening the web UI, starting a session	20s

The single most important instruction in either document

At PART 4, when `[prereq_redirect]` appears, **stop scrolling and leave it on screen** for the whole PART. That line is the entire project. The most common way to ruin this video is to scroll past the best moment in it.

Total timing

PARTs 1 to 9 add up to about **4 minutes 15** of speech. The rulebook allows 3 to 5 minutes, so you have roughly 45 seconds of slack for pauses and for the program thinking. That is comfortable - but not enough to also include PART 10 unless you are running fast. **Going over five minutes is worse than leaving PART 10 out.**

5. Narrating alone

You are doing two jobs at once. Choose your approach during rehearsal, not on the day.

Choose one of these two approaches

Approach	How it works	Trade-off
A - Narrate live	Start the command and talk while output scrolls. Mention the pauses: 'that is a live API call'.	Most impressive, but you must keep up with the output and there are real waits while models respond.
B - Run first, then narrate	Let the whole run finish. Then scroll back through the completed output and narrate over it at your own pace.	Much easier alone, and you control the pace completely. The output is still a real run - nothing is fabricated.

Approach B is not cheating

The rulebook forbids fabricating or manipulating **results**. Scrolling back through the genuine output of a genuine run and explaining it is not that - it is how every code walkthrough in the world is done. What would be wrong is inventing output, or claiming something the screen does not show. **If you are recording alone and nervous, take approach B.** A calm, clear video beats a rushed, authentic-but-flustered one.

Recording in segments

You do not have to do this in one take. Record each PART separately, stop, breathe, then start the next. Join them afterwards in any free editor. This is the single biggest stress reducer available to you, and no one can tell from the finished video.

Six delivery habits

- **Point with the cursor** at the line you are describing. It tells the viewer where to look and keeps you anchored to what is really on screen.
- **Pause after the important sentences.** Silence feels much longer to you than to a viewer. Two seconds after 'It does not move' in PART 5 is right.
- **Slow down at PART 1 and PART 4.** Those two carry the whole idea.
- **Do not read in a monotone.** The script is written the way people speak.
- **If you stumble, stop and redo that PART.** A clean retake costs 30 seconds.
- **Never say a thing the screen is not showing.** If in doubt, describe exactly what is visible. That is always safe and always true.

6. Setup - recording day

This is PART 0 of the script, repeated so it is in both documents.

Step 1 - Prove it works before recording anything

```
cd D:\Downloads\BloodCoded
python run.py check      # one real call per AI provider
python run.py live       # the full demo against real models
```

Step 2 - Run it three times, and know what may vary

Run `python run.py live` three times. **Two things must be identical** every time - they are the two the demo actually claims:

Must be identical	What to look for
The learning path	Learning path : recursion -> functions -> recursion
The self-checks	all 7 [PASS] lines, and no [FAIL]

Plenty of other things will differ, and none of them are faults. Measured over three consecutive runs:

- **The exercise titles change.** The AI writes a new exercise each run - that is the point of grounding generation in a model rather than a question bank.
- **The number of steps can change.** The scripted student submits fixed code, and a differently worded exercise may not accept it. In one run a functions answer was graded wrong, and the agent responded with EXPLAIN_DIFFERENTLY and a lower difficulty - an extra loop, and arguably a better demo than the shorter runs.
- **The provider changes.** Groq until its per-minute ceiling, then Google.
- **The duration changes.** 42 to 65 seconds observed.

That variation is the *student* being scripted while the *tutor* is not, which is precisely the claim. **Stop and tell Sohan only if the learning path itself changes, or if any check reports FAIL.**

Step 3 - Screen and audio

- Terminal about 110 characters wide, font large enough to read on a phone.
- Close everything else: notifications, browser tabs, chat apps.
- Dark terminal theme reads better on video than light.
- Record ten seconds and play it back before a real take.
- Keep the script PDF on a second screen or printed - not on the screen you record.

7. If something breaks while recording

The rule

Say what happened, plainly, and keep going. Do not pretend it did not happen and do not talk over it. A recovered failure demonstrates the recovery path we are claiming - genuinely better evidence than a run where nothing went wrong. And if a take is spoiled, just record that PART again.

What you see	What it means	What to say
degraded=True on a line	An AI provider failed, so a built-in template was used	"The provider is rate-limited there, so it fell back to a deterministic template. The tutoring decisions are unaffected - the model does not make those."
provider=google:...	The first provider hit its limit; it failed over automatically	"That is the provider chain failing over live." A feature - mention it.
A long pause	Waiting for a real API response	"That is a live API call - this is not a recording."
An error you do not recognise	Unknown	"Something went wrong there - let me re-run it." Then run 'python run.py verify', which needs no internet.
The learning path came out different	The one thing that should never vary	Stop. Tell Sohan before continuing.

Known quirk worth understanding

Our first AI provider (Groq) has a free limit of about 8,000 tokens per minute, and one full run uses roughly 35,000. So it may hit the limit partway through and switch automatically to the second provider (Google). You will see `provider=google:gemini-3.5-flash-lite` appear. That is not a fault - it is the failover working, and worth pointing out.

8. Questions a judge may ask

Short honest answers beat long ones. **"I do not know, but I can show you where it is in the code" is a perfectly good answer** and far better than guessing.

Question	Your answer
Is the redirect hardcoded?	No. It is graph traversal over live mastery estimates. There is a test that asserts the state transitions, not the printed text.
Why not just use a big LLM?	An LLM has no calibrated model of the student, so it will happily advance someone who is not ready. The LLM proposes; the guard disposes.
Why not just rules, then?	Rules cannot author a novel exercise grounded in a specific misconception and a specific page of notes. That is what the model is for.
Is the sandbox real?	Yes - a separate process with a timeout and a minimal environment. Student code cannot read our API keys; we verified that with a canary.
Is mastery just a counter?	No, it is Bayesian Knowledge Tracing. It models slipping and guessing and yields a confidence signal the policy consumes.
Did you train a model?	We built parameter fitting for the mastery model. It produced a negative result on held-out data, so we ship the literature defaults and report that.
What happens if the AI is down?	It fails over to a second provider. If all fail, it uses a deterministic template and marks the event degraded - and mastery is untouched, because a provider outage is our fault, not the student's.
Did you write this yourselves?	Yes. Two contributors, both human, and the contributor list is checked automatically before every push.
What does not work yet?	There is no login, no dashboard and no diagnostic quiz - a new student starts from a seeded profile, and nothing is saved between visits. The tutoring engine is complete; the product around it is not. We would rather say that than pretend otherwise.
What breaks it?	Seven and a half percent of students with no gap still get an unnecessary detour. It was twenty-five percent before we required evidence.

9. Glossary

Term	Plain meaning
Agent	A program that decides its own next step instead of following a fixed script.
LangGraph	The library we use to define those steps and the paths between them.
Node	One step in that structure - 'diagnose', 'grade', 'adapt'.
State	Everything the program currently knows about this session.
Checkpoint	A saved copy of that state, written to disk, so it can resume.
Interrupt	The program genuinely stopping to wait for the student - not a loop that spins.
Mastery	Our 0-to-1 estimate of whether the student knows a skill.
Confidence	How much evidence that estimate rests on.
BKT	Bayesian Knowledge Tracing - the method behind those two numbers.
Prerequisite	A skill you must know before another one makes sense.
The guard	Predictable code that checks, and can overrule, the AI's suggestion.
RAG	Searching our own notes and giving them to the AI as source material.
Sandbox	The isolated place student code runs, so it cannot harm anything.
SystemFault	A failure that is ours, not the student's. Never changes mastery.
Ablation	An experiment where you remove one part to prove it was doing something.
Deterministic	Same input, same output, every time. No randomness.
Degraded	A step that fell back to a template because a provider failed.

10. What to study, in priority order

You do not need all of this. If you only do the first three rows, you can still record a good video.

Priority	What	Time	How
Do this first	Section 0 - get the code running on your own machine	30 min	Nothing else in this guide works until it does. Do not leave it to recording day.
Essential	Section 1, until you can say the one idea without reading it	30 min	Read it, then explain it out loud to someone
Essential	Watch a run three times, following the lines in section 3	45 min	'python run.py live' with this guide open beside you
Essential	Read the whole script PDF once, out loud	30 min	Out loud matters - silent reading hides the awkward sentences
Essential	Rehearse PARTs 1 to 9 with the screen, twice	1.5 hours	Time it. Aim for 4:15 of speech.
Strongly	Sections 2.3 and 2.4 - the guard and the safety invariant	45 min	The two a judge is most likely to probe
Strongly	The Q&A table in section 8	30 min	Say the answers aloud; do not just read them
Strongly	Section 5 - decide approach A or B	15 min	Decide in rehearsal, not on the day
If time	Section 2.2 - what BKT actually does	45 min	Optional. The one-sentence version is enough.
If time	Open app/mastery/policy.py and read the rules	1 hour	Ordinary Python with comments. Less scary than it sounds.

A final thought

The strongest thing you can do in a technical Q&A; is be straight about the limits. We know what CogniFlow does not do yet - no login, no dashboard, no diagnostic quiz, nothing saved between visits - and saying so plainly makes every other claim more believable, not less. Confidence is knowing where the edges are, not pretending there are none.

Good luck. The engine works. All you have to do is show it honestly.